

## BDMM 2026: HW Answers

*Submit your answers by April 3 if you would like the grade before the exam.*

*Final deadline is April 10, 2026.*

*Please do not copy the task description into this form, only your answers.*

Full Name: Pedro Miguel Gaspar Santos

Student Number: 20250399

HW Group: 18

Your role (chose one): Part D

List the overall transformations to the database **you group** made to the database: e.g. embedding / referencing choices, pattern uses, indexes, any further redesign and restructuring. *[only one person per group needs to answer this for the whole group]*

*[max 450 words]*

### **Overall Database Transformations and Restructuring**

To make sure our database was fast and optimized for the heavy queries we needed to run across all tasks, we moved away from keeping everything in one massive collection. Keeping it all together eats up memory and risks hitting MongoDB's 16MB document limit. So, we split the data into four focused collections: *listings* (our main one for analytics), *hosts*, *reviews*, and *listing\_text*. This easily solved the "unbounded arrays" problem where lists just keep growing forever.

### **Embedding vs. Referencing:**

We took the reviews array out and moved it into its own collection linked by the *listing\_id* (Referencing). We did the exact same thing for huge text blocks that we almost never search by (like the description). This makes the listings collection super light so MongoDB can process it quickly in RAM. However, the attributes we filter by all the time were kept embedded in the listings collection so we don't have to do heavy joins (\$lookup) just to filter data.

## Design Patterns Applied:

1. **Subset Pattern:** So that we wouldn't have to do a slow \$lookup every time we just wanted to see recent feedback, we saved a *reviews\_subset* array with the 5 most recent reviews directly inside listings.
2. **Extended Reference Pattern:** We moved the detailed host info to the hosts collection, but we left key fields (like *host\_is\_superhost*) copied in listings to filter things fast.
3. **Computed Pattern:** This was our biggest optimization. To avoid doing the same math repeatedly in our queries, we calculated complex metrics upfront during the ETL phase. We added *price\_ppn\_2g*, *host\_type*, and *popularity\_score* directly to *listings*, and we saved the *avg\_rating* in the hosts collection.
4. **Attribute Pattern:** Instead of creating dozens of indexes for every single host feature, we grouped those boolean options into a *host\_traits* key-value array. It makes searching for features way easier.
5. **Outlier Pattern:** We flagged the "mega-hosts" (the top 5% with the most listings) right away so we can easily take them out of our normal statistics.
6. **Schema Versioning Pattern:** We added a *schema\_version: 2* tag to all modernized documents. This prepares the database for the future, allowing the application to easily distinguish between the old and new structure.

## Indexing & Data Cleaning:

We created targeted indexes to back up these patterns. We used compound indexes on listings for location and price, a multikey sparse index (*idx\_host\_traits\_kv*) to search host features in a flash, and a partial index (*idx\_pro\_popularity*) just for professional hosts, which saves a ton of memory.

Before splitting the database we converted Decimal128 values to normal floats (to make math easier in Python and MongoDB), set empty numeric fields to 0.0, and deleted the duplicates.

### Question 1. Assumptions [max 350 words]

Before diving into the pricing analysis, I had to make a few important decisions to ensure our data was clean, realistic, and fair.

First, I needed a solid way to classify hosts. A professional host is someone managing more than one listing in our dataset, while an amateur manages exactly one. I calculated this by directly counting the listings per *host\_id* in our database. I deliberately avoided using the

existing *host\_total\_listings\_count* field because it often includes properties from other cities or platforms, which would misrepresent their actual presence in our specific market sample.

Next, I had to calculate a realistic price. The instructions asked for the effective price per person per night for a two-guest, one-week stay. We created a custom computed field, *price\_ppn\_2g*. The formula we used was:  $(\text{price} * 7 + \text{cleaning\_fee} + \max(0, 2 - \text{guests\_included}) * \text{extra\_people} * 7) / 14$ . We made sure to include the nightly rate, the one-time cleaning fee, and any extra-person surcharges if the base price only covered one guest. However, we specifically excluded the security deposit, as this is fully refundable and not an actual cost to the guest.

During our data cleaning, we noticed that 1,681 listings had a completely missing price (None). I excluded these from Task D analysis so they wouldn't break our aggregations or skew the distributions (these are handled separately in Task A3).

Finally, real estate markets always have extreme luxury outliers. To prevent a few ultra-expensive mansions from completely compressing and distorting our box plots, I clipped the prices at the 95th percentile for each specific city (outlier pattern). This allows us to visualize the true "typical" market distribution.

## Question 1. Visual

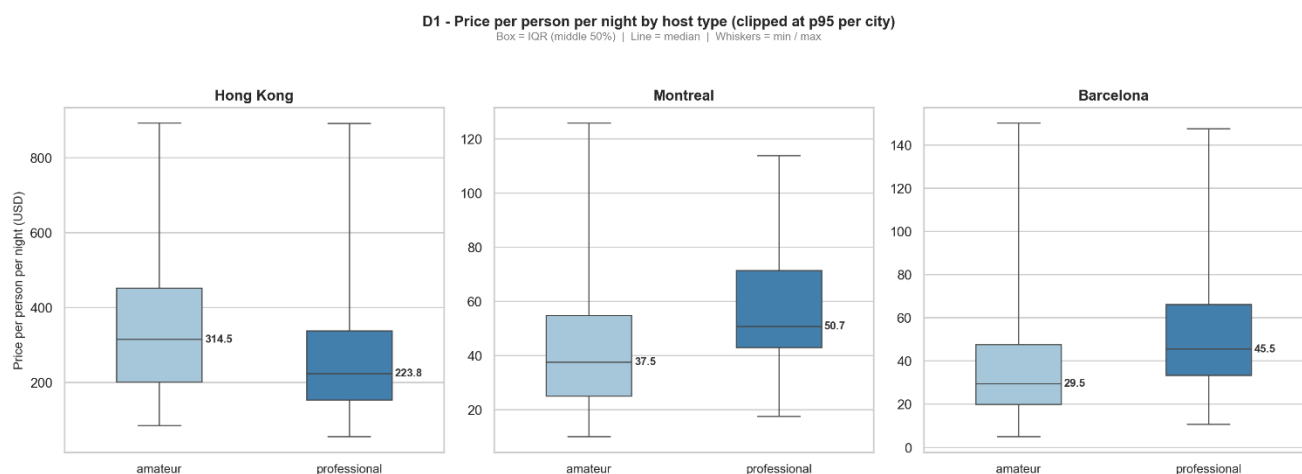


Figure 1 - Price per person per night by host type

### Question 1. Interpretation *[max 350 words]*

Looking at the box plots, we can see very distinct pricing strategies depending on the city and the host type.

In Montreal and Barcelona, professional hosts charge noticeably higher prices than amateurs. In Montreal, the median price per person per night for professionals is around \$50.70 compared to \$37.50 for amateurs. In Barcelona, the gap is similar, at \$45.50 versus \$29.50. This premium makes perfect sense when we look at their portfolio composition: professionals in these cities almost exclusively manage "Entire home/apt" listings (about 94% of their portfolio in Montreal and 88% in Barcelona), which naturally command higher market prices than private rooms.

However, Hong Kong presents a fascinating reversal. At first glance, amateurs appear to charge significantly more than professionals, with a median of \$314.50 compared to \$223.80 for professionals. This seemed counterintuitive, so I investigated further and discovered a classic composition effect (a real-world example of Simpson's Paradox). The real estate market in Hong Kong operates differently: professional hosts there actually focus heavily on smaller, cheaper units. Only 30% of their listings are "Entire homes", compared to 56% for amateur hosts.

To prove this was just a composition effect and not a true difference in underlying pricing strategy, I controlled our data to compare only entire-home listings in Hong Kong. Once I compared apples to apples, the massive pricing gap almost completely vanished (amateurs at roughly \$418.40 versus professionals at \$392.60). This confirms that professionals in Hong Kong do not systematically underprice amateurs - they simply operate in a different accommodation segment.

Finally, regarding market spread, Hong Kong exhibits the widest price variability, reflecting a highly diverse market. In contrast, Montreal's pricing is the most tightly compressed. It is also worth noting that I applied a 95th percentile clip to remove extreme luxury outliers (about 5% of the data) so the main distributions in our box plots remain readable, though raw statistics were preserved for full transparency.

### Question 2. Assumptions *[max 350 words]*

To measure how specific host features impact guest satisfaction, I used the *review\_scores\_rating* on a 0-100 scale. Listings without a rating (approximately 1472 out of 5553) were excluded from this analysis entirely, as including them would introduce noise into the satisfaction averages. Since this task requires a host-level analysis rather than a

listing-level one, I used the Computed Pattern during our ETL phase to pre-aggregate the *avg\_rating* across all listings for hosts managing multiple properties. This means each host contributes a single average score regardless of how many listings they manage.

Next, I mapped five features into strict binary values. *is\_superhost* and *is\_identity\_verified* were straightforward as they were already boolean. For the others, we defined *is\_fast\_responder* as hosts responding "within an hour" or "within a few hours", *is\_perfect\_responder* as having exactly a 100% *host\_response\_rate* and *is\_flexible\_cancellation* as having a "flexible" or "moderate" policy.

Crucially, I implemented strict null handling. If a host had missing data for any of these features, I explicitly excluded them from that specific True/False split instead of defaulting them to False. This prevents missing data from inflating the False group, which is exactly what happened in the naive pipeline before we corrected it. Finally, I decided not to apply artificial weights to the satisfaction difference score, simply calculating it as the raw unweighted difference between the True and False group averages.

Question 2. Visual of data

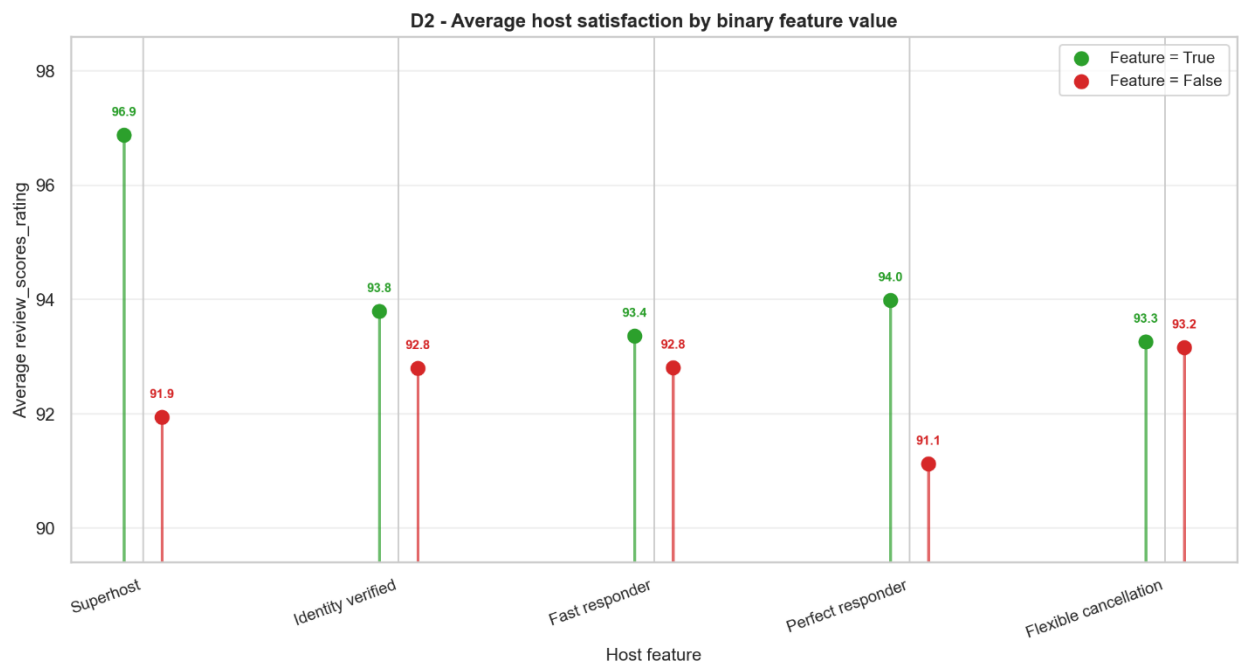


Figure 2 - Average host satisfaction by binary feature value

Question 2. Visual of query before and after

|                | Before (naive, listings) | After (optimised, hosts) |
|----------------|--------------------------|--------------------------|
| Scan type      | -                        | IXSCAN                   |
| Index used     | none                     | idx_hosts_avg_rating     |
| Docs examined  | 5553                     | 3797                     |
| Keys examined  | 0                        | 3798                     |
| Returned       | 3797                     | 3797                     |
| Exec time (ms) | 56                       | 13                       |
| Wall time (ms) | 33.9                     | 27.5                     |

Figure 4 - Visual of query before and after table (naive -> optimized (more efficient))

| Naive (listings, in-flight mapping) \            |                      |
|--------------------------------------------------|----------------------|
| Scan type                                        | -                    |
| Index used                                       | none                 |
| Docs examined                                    | 5553                 |
| Keys examined                                    | 0                    |
| Returned                                         | 3797                 |
| Exec time (ms)                                   | 56                   |
| Wall time (ms)                                   | 33.9                 |
| Computed (hosts, top-level fields) \             |                      |
| Scan type                                        | IXSCAN               |
| Index used                                       | idx_hosts_avg_rating |
| Docs examined                                    | 3797                 |
| Keys examined                                    | 3798                 |
| Returned                                         | 3797                 |
| Exec time (ms)                                   | 13                   |
| Wall time (ms)                                   | 27.5                 |
| Attribute (hosts, host_traits + multikey sparse) |                      |
| Scan type                                        | IXSCAN               |
| Index used                                       | idx_hosts_avg_rating |
| Docs examined                                    | 3797                 |
| Keys examined                                    | 3798                 |
| Returned                                         | 3797                 |
| Exec time (ms)                                   | 17                   |
| Wall time (ms)                                   | 18.2                 |

Figure 3 - Visual of query before and after detailed (naive -> computed -> attribute (simpler query))

## Question 2. Interpretation *[max 450 words]*

Looking at the satisfaction differences, Superhost status is by far the strongest predictor of guest satisfaction, showing a massive +5-point advantage (97 vs 92). Maintaining a perfect response rate (100%) is the second-strongest signal, yielding a +2.9-point gap. Identity verification (+1.0) and fast response times (+0.6) show modest positive effects. Surprisingly, offering a flexible cancellation policy has almost zero impact (+0.1), suggesting that while guests might appreciate it during booking, it doesn't meaningfully influence the final review scores they leave.

On the database engineering side, performance optimizations were highly successful. I initially built a naive pipeline that ran directly on the listings collection. This required a slow COLLSCAN on 5,553 documents, calculating the binary mappings and grouping the data in-flight. Notably, this naive approach also incorrectly counted null values as 'False', artificially inflating the False group.

To fix this, I built an optimized pipeline querying the pre-computed hosts collection. By leveraging the Computed Pattern fields and an IXSCAN via the `idx_hosts_avg_rating` index, the optimized pipeline correctly excluded nulls. This optimization reduced the number of documents examined by 32% (from 5,553 to 3,797) and cut the execution time from 56 ms to 13 ms (important: these were the values I got at the moment of writing this report – they can change).

Furthermore, I evaluated the Attribute Pattern (the `host_traits` array) against the standard top-level fields approach. While querying top-level fields is slightly faster for a single isolated query, the Attribute Pattern paired with a multikey sparse index makes our database infinitely more scalable. It allows us to answer any trait-based question with just one index, avoiding the need to build new dedicated indexes as we track more host traits over time.

## Question 3. Assumptions *[max 350 words]*

To analyze the relationship between popularity and satisfaction for professional hosts, I first had to define a reliable Popularity Score. I decided to use property unavailability as a proxy for demand, assuming that for professional hosts, a blocked calendar means a booking. This was calculated this by looking at four time horizons (30, 60, 90, and 365 days) and applying a weighted average to the unavailability:

- 30-day (40%): This has the highest weight because it is the strongest indicator of current, real-time demand.
- 60-day (25%), 90-day (20%), and 365-day (15%): These capture longer-term trends.

These overlapping windows were chosen intentionally; a near-term booking contributes to all four terms, which correctly gives it more "power" in the final score (clamped between 0 and 1).

Regarding satisfaction, the dataset is heavily skewed towards high scores (median of 95). To create meaningful groups, I defined:

- Low (< 80): Properties clearly underperforming relative to the market.
- Medium (80-94): Properties performing around or slightly below the median.
- High (>= 95): Top-tier properties.

I also decided to identify the three most common property types among professionals dynamically (Apartments, Condominiums, and Serviced apartments) to ensure my analysis focused on where the data is most representative.

Finally, I used the Computed Pattern to store the *popularity\_score* on each listing for periodic refreshes. To make the recurring analysis efficient, I implemented a Partial Index (*idx\_pro\_popularity*) that only targets professional hosts with valid scores, significantly reducing the work the database has to do.

### Question 3. Visual

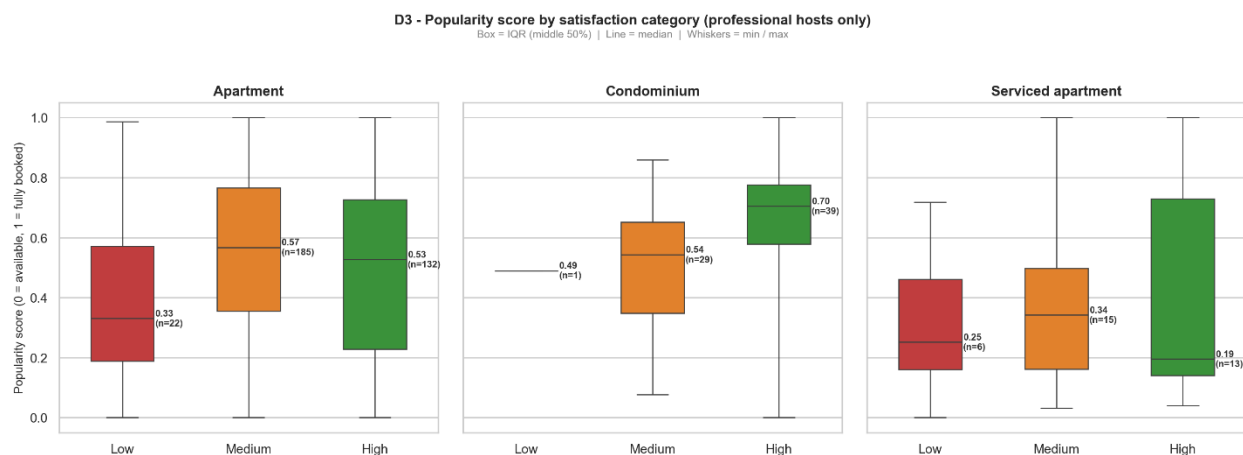


Figure 5 - Popularity score by satisfaction category



### Question 3. Interpretation *[max 450 words]*

The findings show that while satisfaction influences popularity, the impact varies significantly depending on the type of property.

Condominiums follow the most predictable trend: as satisfaction increases, so does popularity. The median popularity score rises from Low (0.49, though based on just n=1) through Medium (0.54, n=29) to High (0.70, n=39). Despite the unreliable Low cell, the Medium-to-High progression is clear and suggests that for this segment, maintaining a high reputation is directly linked to higher occupancy rates.

Apartments, the largest sample (n=339), reveal a more complex story. Interestingly, "Medium" satisfaction properties actually show the highest median popularity (0.57), even slightly higher than the "High" tier (0.53). This suggests a "plateau" effect: for standard apartments, once you reach a solid satisfaction level, you already drive maximum demand. The absolute top-tier apartments might represent newer or more premium listings that, despite perfect scores, haven't yet reached full occupancy or are priced in a way that limits total bookings.

Serviced apartments showed no clear trend, with the "Medium" group leading (0.34). However, we must be careful with this interpretation, as the small sample sizes in this group limit our statistical confidence.

A key takeaway across all groups is the wide variability of the scores. In almost every category, the interquartile range (IQR) spans 0.2 to 0.4 units. This tells us that while guest satisfaction is important, it isn't the only driver of success. External factors like location, seasonality, and pricing strategy are likely just as influential. Price per person per night was pre-calculated during ETL but was not included as a variable in this analysis, which focuses strictly on the satisfaction-popularity relationship.

From a technical standpoint, the use of the Computed Pattern and the Partial Index proved essential. By pre-calculating the popularity score during ETL and using a targeted index, I successfully avoided full collection scans, ensuring that this analysis remains fast and scalable as the database grows with new availability data.